

SSGMCE/PRM/52-B		SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
SSGMCE	PRACTICAL EXPERIMENT INSTRUCTION SHEET				
	EXPERIMENT TITLE :				
EXPERIMENT NO. : SSGMCE/WI/IT/6IT09/07			ISSUE NO. : 00	ISSUE DATE : 01-01-2026	
REV. DATE : NA		REV. NO. : 00	DEPTT. :	INFORMATION TECHNOLOGY	
LABORATORY : COMPUTER SKILL LAB IV				SEMESTER : VI	PAGE :1 OF 4

01 AIM: Write a program to Implement a Text Classifier for Spam Vs Not-Spam Using Naïve Bayes Classifier

02 SCOPE: To understand and implement a Text Classifier for Spam Vs Not-Spam Using Naïve Bayes Classifier. The model helps in SMS spam detection and used in email filtering systems. It is also applicable in text classification problems in AI/ML and useful in cybersecurity and fraud detection.

03 FACILITIES:

Hardware: Computer System with Python and with minimum 4GB RAM.

Software: Python (version 3.x)

Libraries: pandas, scikit-learn

Dataset: spam.csv (SMS Spam Collection dataset)

04 THEORY:

1. Introduction to Text Classification

Text classification is one of the most important applications of Artificial Intelligence and Machine Learning, where textual data is automatically categorized into predefined classes. In everyday life, we encounter text classification in email filtering, sentiment analysis, news categorization, and spam detection systems.

In this experiment, we focus on spam detection, where the goal is to classify a given message as either:

Spam (unwanted or promotional message)

Not Spam (Ham) (normal or useful message)

The challenge lies in the fact that computers do not understand human language directly. Therefore, we need techniques to convert text into a format that machines can process efficiently.

2. Naïve Bayes Algorithm

Naïve Bayes is a supervised machine learning algorithm based on probability theory, specifically Bayes' Theorem. It is widely used for classification tasks, especially when dealing with large amounts of text data.

The algorithm calculates the probability that a given message belongs to a particular class (Spam or Not Spam) based on the words present in that message. It uses the following idea:

$P(\text{Spam} | \text{Message})$

This means: What is the probability that the message is spam, given its content?

The term "Naïve" comes from the assumption that all words in a message are independent of each other, which is not always true in real life. However, this assumption simplifies the computation and still provides very good results in practice.

3. Concept of Vectorization:

Machines cannot understand words or sentences directly. Therefore, we need to convert textual data into numerical form, which is called vectorization.

Vectorization transforms each message into a vector (a list of numbers), where each number represents the occurrence of a word in the message. This allows machine learning algorithms to process and analyze the data effectively.

4. Count Vectorizer:

In this experiment, we use CountVectorizer, a simple and widely used technique for text vectorization.

CountVectorizer works by:

Identifying all unique words in the dataset

Assigning each word a position in a vector

Counting how many times each word appears in a message

For example:

Message	Representation
"win money"	[1, 1]
"hello friend"	[1, 1]

Each column corresponds to a word, and the values indicate how frequently the word appears. This method is simple, efficient, and sufficient for basic text classification tasks like spam detection.

5. Workflow of the System

The overall working of the system can be understood in the following steps:

1. The dataset containing labeled messages is loaded.
2. The labels (Spam/Ham) are converted into numerical form (1/0).
3. The dataset is divided into training and testing sets.
4. Text data is converted into numerical vectors using CountVectorizer.
5. The Naïve Bayes model is trained using the training data.
6. The trained model is tested on unseen data to evaluate its performance.
7. Finally, the system accepts a new message from the user and predicts whether it is Spam or Not Spam.

6. Real-World Importance:

Spam detection is a crucial application in today's digital world. Every day, millions of unwanted messages are sent through emails and mobile networks. Machine learning models like Naïve Bayes help in:

Reducing spam messages

Improving user experience

Enhancing security and productivity

This experiment provides a basic understanding of how such intelligent systems are designed and implemented.

05 PROGRAM:

```
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
data = pd.read_csv("spam.csv", encoding='latin-1')

# Select required columns
data = data[['v1', 'v2']]
data.columns = ['label', 'message']

# Convert labels to numeric
data['label'] = data['label'].map({'ham': 0, 'spam': 1})

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    data['message'], data['label'], test_size=0.2, random_state=42
)

# Vectorization
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Train model
model = MultinomialNB()
model.fit(X_train_vec, y_train)

# Predict
y_pred = model.predict(X_test_vec)

# Performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# User input
msg = input("\nEnter a message: ")
msg_vec = vectorizer.transform([msg])
result = model.predict(msg_vec)[0]

# Output result
if result == 1:
    print("SPAM")
else:
    print("NOT SPAM")
```

05 SAMPLE OUTPUT:

 Model Performance-----
Accuracy: 0.9838565022421525

Classification Report:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	965
1	0.99	0.89	0.94	150
accuracy			0.98	1115
macro avg	0.98	0.95	0.96	1115
weighted avg	0.98	0.98	0.98	1115

 Try Your Own Message-----
Enter a message: Lucky Winner of 10000 rupees. Reply with Bank Account Number to credit the prize money.

Result: 🚫 SPAM

06 Conclusion:

Here we have successfully implemented a Spam Classifier using Naïve Bayes and it is observed that we achieved high accuracy (~95–98%). Also, we learned to: Convert text into numerical form, Train and evaluate a machine learning model and perform real-time prediction using user input. This experiment demonstrates how machine learning can be used in real-world applications like spam detection.